RAG and Evidence Support

Comparing TF-IDF and Embedding Retrieval on an Open ML Corpus

CS420 — Introduction to Artificial Intelligence
Seminar S07

Phan Anh Khoa — 24125059
Bùi Trọng Thành — 24125043

Abstract

This report studies retrieval-augmented generation for document question answering. Drawing on Lewis et al. [1] and Schneider's discussion of verifiability [2], it distinguishes retrieval success, answer correctness, and evidence support. The corpus is the seven-page abess paper, published in 2022 under CC BY 4.0, with 20 English questions and selected reference spans. TF-IDF and MiniLM use identical chunks of 96 or 192 tokens and $k \in \{1, 3, 5\}$. At 192 tokens and $k = 3$, macro chunk Recall@k is 0.80 for TF-IDF and 0.60 for MiniLM. Across 60 Gemini responses, assisted review identifies 5/20 fully correct closed-book answers, 16/20 with TF-IDF, and 14/20 with MiniLM. No RAG answer was scored incorrect, against 9/20 closed-book, although RAG responses still abstained or went beyond explicit evidence. Retrieval recall does not measure answer accuracy, and neither a small corpus nor incomplete relevance labels supports broad claims about model reliability.

1 Introduction

An LLM can produce fluent text without evidence that a reader can verify. Document question answering also requires source identity, document version, and the scope of each claim. RAG retrieves selected passages before generation and supplies them as context, allowing answers to refer back to identifiable sources.

Research question: Does supplying external documents eliminate hallucination? Two related questions guide the demonstration: how do representation and chunk size affect evidence retrieval, and does the retrieved evidence actually support the generated answer?

This is a small, reproducible demonstration of probabilistic formulations and failure mechanisms. It does not retrain the LLM, reproduce the original paper's Wikipedia-scale system, or establish a general ranking of models. The corpus is a 2022 snapshot, so questions about versions and benchmarks refer to that historical paper.

Published results from the foundational papers are kept separate from measurements produced by this project. Reference answers are explicitly labeled and never substituted for API responses. Every question is answerable from the full corpus, but its answer need not appear in a particular retriever's top-k context.

2 Background and Paper Review

Parametric memory is knowledge represented in learned model weights. **Non-parametric memory** is an external store accessed through retrieval. The latter describes where knowledge is stored; query and document encoders can still have learned parameters.

Approach	Knowledge location	At answer time	Updating
Fine-tuning	Model weights	Use adapted model	Usually train again
Full-document prompt	Input context	Read supplied documents	Supply a new version
RAG	External store	Retrieve top-k, then generate	Update store and index

Table 1: Project synthesis based on [1]. These approaches can be combined.

Lewis et al. combine DPR with BART, train the query encoder and generator, and keep the document encoder and index fixed. Their December 2018 Wikipedia snapshot contains about 21 million passages of 100 words each. Documents are latent variables, and output probabilities are marginalized over retrieved documents. Our TF-IDF/MiniLM/Gemini pipeline is a different implementation.

The paper introduces a combination of parametric and non-parametric memory for knowledge-intensive tasks, with RAG-Sequence and RAG-Token formulations. Its Table 1 reports Natural Questions Exact Match of 44.5 for RAG-Sequence versus 41.5 for DPR. Its Table 6, a dev-set ablation, shows BM25 slightly ahead of the learned dense retriever on FEVER (3-way accuracy 75.1 versus 74.5), while the authors report that differentiable retrieval performs better on the other tasks. Dense retrieval should therefore not be assumed to win universally, although this single exception is small. These are the authors' published results, not measurements from this project.

Schneider [2] reviews explainability for generative AI, including *verifiability*, meaning that explanations should support verification of outputs, and *lineage*, meaning the tracking of data, algorithms, and processes across a model's lifecycle. We apply lineage only by analogy, recording the origin of each retrieved passage. Schneider notes that RAG facilitates verifying an LLM's response, and that model self-explanations are not necessarily faithful. RAG narrows the sources to inspect without proving that they support every generated claim. Lewis provides a model and empirical evidence; Schneider motivates scrutiny of explanations and their origins.

Paper	Relevant contribution	Application in this project
Lewis et al. [1]	Retrieval and generation with Sequence/Token marginalization	Theoretical foundation; our API pipeline concatenates context and does not implement these marginalizations.
Schneider [2]	Verifiability of generated outputs; lineage of AI models and data	Inspect source provenance and whether each cited passage supports the associated claim.

Table 2: Synthesis of the readings and their roles in the project.

The PDF is extracted once and split into identical chunks for both methods. At query time, each method converts the question into its own vector space, scores all chunks by cosine similarity, and

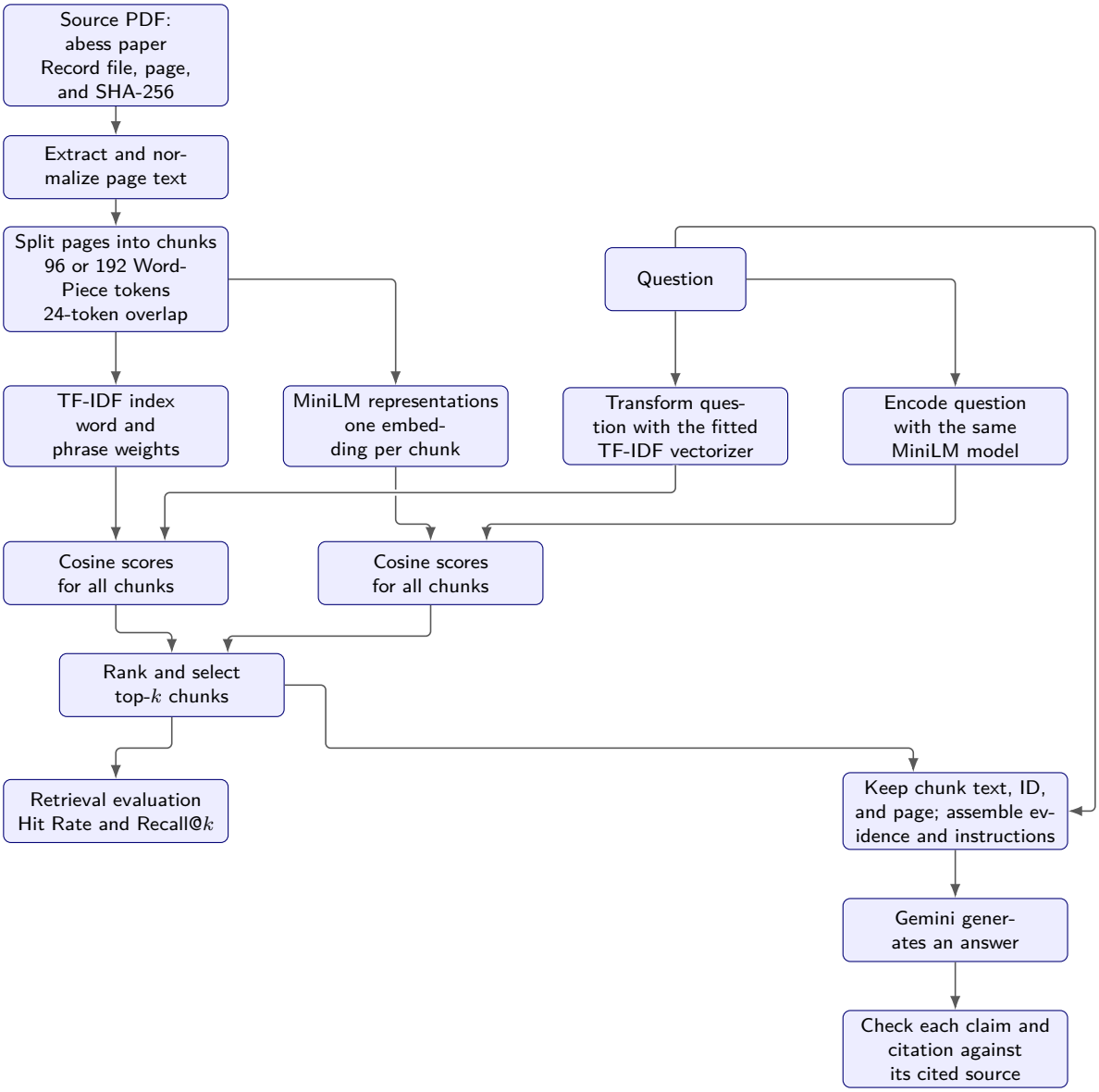


Figure 1: Project pipeline informed by Lewis et al. [1]. Each RAG answer uses one retriever's top-k chunks. The closed-book comparison sends the question to Gemini without retrieved context.

selects top-k. We evaluate those selected chunks against reference evidence. For answer generation, the saved chunks from one method are inserted into a source-labeled prompt; the resulting claims and citations are checked against the cited text. This is prompt-based RAG, not the probability marginalization of RAG-Sequence or RAG-Token.

3 Method and Worked Example

RAG-Sequence and RAG-Token

Let x be the input question, $y = (y_1, \ldots, y_N)$ the output token sequence, and $Z_k(x)$ the retrieved top-k set. RAG-Sequence is [1]:

$$p(y \mid x) \approx \sum_{z \in Z_k(x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z), \qquad p_\theta(y \mid x, z) = \prod_{i=1}^{N} p_\theta(y_i \mid x, z, y_{<i}). \tag{1}$$

Here z is a latent document, k is a positive integer no larger than the number of passages, N is the output length, and $y_{<i}$ contains the preceding output tokens. η and θ denote retriever and generator parameters. Probabilities are dimensionless and lie in $[0, 1]$. Restricting the sum to top-k approximates a sum over the full store; an implementation must specify whether it renormalizes retrieved weights.

RAG-Sequence holds z fixed for the entire output within each term of the sum. RAG-Token marginalizes the document separately at each token position:

$$p(y \mid x) \approx \prod_{i=1}^{N} \sum_{z \in Z_k(x)} p_\eta(z \mid x) p_\theta(y_i \mid x, z, y_{<i}).$$

This does not require a new database search for every token. Concatenating passages into an API prompt, as our demo does, does not implement either marginalization formula.

Hypothetical numerical example. Assume two documents form the entire support, with normalized weights $p_\eta(z_1 \mid x) = 0.6$ and $p_\eta(z_2 \mid x) = 0.4$. Suppose the probabilities of the *complete sequence* y conditioned on those documents are 0.7 and 0.2. First compute $0.6 \times 0.7 = 0.42$, then $0.4 \times 0.2 = 0.08$. Their sum is $p(y \mid x) = 0.50$. These numbers illustrate the equation; they were not obtained from Gemini. A sequence probability is not the probability that its factual content is true.

Hypothetical RAG-Token comparison. Use the same weights and a two-token output, with $p_\theta(y_1 \mid x, z) = 0.875, 0.5$ and $p_\theta(y_2 \mid x, z, y_1) = 0.8, 0.4$ for z_1, z_2; these factors reproduce the sequence probabilities 0.7 and 0.2 above. RAG-Token mixes documents at each position: $0.6 \times 0.875 + 0.4 \times 0.5 = 0.725$ for y_1 and $0.6 \times 0.8 + 0.4 \times 0.4 = 0.64$ for y_2, so $p(y \mid x) = 0.725 \times 0.64 = 0.464$. RAG-Sequence gives 0.50 because each term keeps one document for the whole output. These values are also illustrative.

Representations and Ranking

TF-IDF uses lowercase text, English stopwords, unigrams/bigrams, and scikit-learn's smoothed IDF [4]. For M chunks, document frequency $df(t)$, and raw term count $tf(t, d)$:

$$idf(t) = \log \frac{1 + M}{1 + df(t)} + 1, \qquad w_{t,d} = tf(t, d) idf(t).$$

IDF is dimensionless and at least 1. Questions use the vocabulary and IDF fitted on corpus chunks; questions and answers are not used to fit them. Dense retrieval uses 384-dimensional all-MiniLM-L6-v2 embeddings [5]. Both methods L2-normalize vectors and compute:

$$\cos(q, d) = \frac{q^\top d}{\|q\|_2 \|d\|_2}.$$

Cosine lies in $[-1, 1]$ generally and in $[0, 1]$ for nonnegative TF-IDF vectors. A zero TF-IDF query receives similarity 0. For $q = (1, 1)$ and $d = (1, 0)$, cosine is $1/\sqrt{2} \approx 0.707$. Higher similarity does not imply a calibrated probability of correctness. The small corpus permits exhaustive search without an approximate vector index.

Corpus and Evaluation Design

The corpus contains all seven pages of Zhu et al. [3]: pages 1–4 contain the main text, and pages 5–7 contain acknowledgments and references. Page 1 states CC BY 4.0. A manifest records the source URL, authors, license, and SHA-256 hash. Extraction applies Unicode NFKC normalization, line-end dehyphenation, and whitespace collapse. Tables are not rewritten into answer-bearing prose.

Twenty questions and reference answers were saved before ranking, with evidence spans on specified pages. Q09 distinguishes article and software licenses, Q18 checks the scope of polynomial-time guarantees, and Q19 challenges a universal interpretation of a speedup claim. Questions and references are neither indexed nor used for training. AI-assisted gold annotations have been checked against the source text.

Page-wise chunking uses MiniLM's WordPiece tokenizer: 96 or 192 tokens with 24-token overlap, producing 81 or 36 chunks. Including special tokens, every chunk fits the model's 256-token limit. No long chunk is silently truncated. Each method uses the same chunks and $k = 1, 3, 5$, with stable tie ordering. MiniLM tokens differ from words, characters, and Gemini billing tokens.

Recall at top-k (Recall@k) is ordinary recall restricted to the first k ranked results [7]. Let R_q contain chunks enclosing at least one complete selected gold span, and $T_{q,k}$ be the retrieved top-k for question q:

$$\text{Recall@k}(q) = \frac{|R_q \cap T_{q,k}|}{|R_q|}, \qquad \text{MacroRecall@k} = \frac{1}{20} \sum_q \text{Recall@k}(q).$$

Every R_q is nonempty. Scores are dimensionless and lie in $[0, 1]$. Overlap can create multiple relevant chunks containing the same fact. We therefore also report evidence recall (fraction of gold spans fully covered by the union of returned chunks), hit (at least one span covered), and all-evidence (every span covered). Retrieving one of two required spans gives evidence recall $1/2$, hit 1, and all-evidence 0.

Gold spans are selected references rather than exhaustive relevance labels. Q03's PyPI/CRAN information also appears on page 2, outside its selected page-1 gold. Retrieval scores therefore cannot replace answer review. Generation compares closed-book, TF-IDF RAG, and MiniLM RAG using Gemini 3.5 Flash-Lite [6], temperature 0, and at most 2048 output tokens. Every prompt asks for an answer of at most 100 words, which can limit detail. Calls are independent. Both RAG conditions use 192-token chunks and $k = 3$, selected before inspecting results.

Question, Retrieved Context, and Answer

Question (Q08). At the time of writing, what were the Python and R package test coverages?
Retrieved evidence. "The coverages for the Python and the R packages at the time of writing are 97% and 96%, respectively."
Source: abess [3], page 2; `s192-p2-t336`. This is an excerpt of the retrieved chunk, not the entire context.
Prompt context. TF-IDF supplies `s192-p2-t336`, `s192-p1-t336`, and `s192-p2-t504`. Instructions require using supplied evidence, citing it, and acknowledging gaps.
Reference answer. "The paper reports 97% test coverage for Python and 96% for R [s192-p2-t336]."
Actual Gemini answer. "At the time of writing, the code coverages for the Python and the R packages were 97% and 96%, respectively [s192-p2-t336]."
Verification. Both numbers, their Python/R ordering, the historical qualifier, and the cited chunk match the evidence. The answer is copied verbatim from the Q08/`tfidf` record in `results/generation.jsonl`.

4 Results and Discussion

Measured Retrieval Results

Method	Tokens	k	Hit	Recall@k	Evidence	All
tfidf	96	1	0.500	0.425	0.500	0.500
tfidf	96	3	0.700	0.625	0.650	0.600
tfidf	96	5	0.800	0.750	0.775	0.750
embedding	96	1	0.250	0.200	0.250	0.250
embedding	96	3	0.500	0.425	0.475	0.450
embedding	96	5	0.650	0.550	0.600	0.550
tfidf	192	1	0.500	0.450	0.500	0.500
tfidf	192	3	0.850	0.800	0.800	0.750
tfidf	192	5	0.900	0.900	0.900	0.900
embedding	192	1	0.450	0.400	0.450	0.450
embedding	192	3	0.650	0.600	0.625	0.600
embedding	192	5	0.800	0.725	0.750	0.700

Table 3: Results on 20 questions. Hit/All: fraction of questions with at least one/every gold span covered; Recall@k: macro chunk recall; Evidence: evidence recall. Source: project data.

At 192 tokens and $k = 3$, TF-IDF achieves 85% hit, 80% evidence recall, and 75% all-evidence. MiniLM achieves 65%, 62.5%, and 60%, respectively. At $k = 5$, evidence recall rises to 90% and 75%. These are retrieval measurements. Generation was not run at every k, so these increases do not establish improved answer quality.

Questions contain class names, license terms, NNZ, and AUC that overlap with the paper's vocabulary, plausibly favoring TF-IDF. The small MiniLM model sometimes retrieves broadly related software passages while missing exact details. This is a post-hoc interpretation, not a causal separation of model, wording, and preprocessing effects. The result does not show that lexical retrieval generally outperforms dense retrieval.

Three Retrieval Failures

Q06: missing evidence. With 192-token chunks and $k = 3$, TF-IDF misses the passage specifying Python ≥ 3.6 and R $\geq 3.1.0$. MiniLM retrieves `s192-p2-t336`. A method with a lower aggregate score can still succeed on a particular question. Without the required context, distinguish abstention from guessing or use of outside knowledge.

Q13: related passages that do not answer the question. TF-IDF retrieves the conclusion and references instead of the architecture passage explaining extension through an Algorithm subclass. MiniLM retrieves `s192-p2-t672`. A passage about machine-learning tasks does not necessarily explain how to extend the library.

Q09: evidence with multiple parts. The answer requires GPL-3 for the code and CC BY 4.0 for the article. TF-IDF retrieves the page-2 GPL-3 statement but misses the article license on page 1. A hit-only score could suggest success despite the incomplete answer. All-evidence and claim-level review expose the gap. A possible improvement is to issue subqueries for each object being compared.

Answer Quality and Citation Review

The final run contains 60 responses from Gemini 3.5 Flash-Lite, with 20 per condition and temperature 0. All conditions receive the same paper title, authors, and year. Both RAG conditions use 192-token chunks and $k = 3$. This matched-metadata run follows a pilot whose closed-book prompt could leave the paper identity ambiguous. Pilot outputs are stored separately and excluded below.

Condition	Fully correct	Partial	Incorrect	Abstained	Mean score
Closed-book	5/20	6/20	9/20	0/20	0.400
TF-IDF RAG	16/20	2/20	0/20	2/20	0.850
MiniLM RAG	14/20	3/20	0/20	3/20	0.775

Table 4: AI-assisted review of actual responses, not an independent blind human evaluation. Scores: complete 1, partial 0.5, wrong/abstained 0. The four outcome categories are mutually exclusive and sum to 20 per condition. Incorrect means a zero-score answer that did not abstain; both incorrect answers and abstentions receive 0.

The rubric checks the target entities, numbers, and qualifications in the reference. Each label has a rationale and response hash in `answer_review.csv`. Partial credit involves judgment: Q19 identifies the cancer-specific scope but still states that information is insufficient instead of explicitly rejecting a universal claim. Phan Anh Khoa (24125059) and Bùi Trọng Thành (24125043) each reviewed and accepted the assisted evaluation without requesting score changes. This acceptance is not a blind re-scoring exercise.

Context correcting an error. For Q16, closed-book returns 100 test sets, whereas the PDF states 20. Both RAG responses return 20 and cite `s192-p4-t504`. For Q06, closed-book incorrectly gives R 3.5.0; MiniLM correctly gives R $\geq$ 3.1.0; TF-IDF abstains because the necessary passage is absent. Abstention receives no correctness credit even when it is appropriate for the supplied evidence.

Correct content with a missing citation. MiniLM's Q14 response correctly lists `print, coef,`

`plot` but gives no citation. Among the 40 final RAG responses, the structural check finds no extracted chunk ID outside the supplied set. This does not establish semantic support or detect every possible malformed citation. For Q07, MiniLM infers the code-checking service from a GitHub Actions URL, while the context lacks an explicit continuous-integration statement; the review records this as inference from a URL.

Gold coverage and context use. MiniLM answers Q03 correctly from page-2 PyPI/CRAN URLs, although the selected gold is on page 1. For Q15 it abstains on AUC even though a chunk begins with 0.9663829492654472 near the Figure 3 caption. The code example is split, and the response mistakenly explains the gap in terms of the benchmark table. These cases show why actual context must be read rather than inferring response quality from selected-span recall.

Abstention behavior. Although the closed-book prompt says to admit when the answer is unknown, it abstained on 0/20 questions and gave specific wrong values such as R 3.5.0 and 100 test sets. RAG responses abstained 5/40 times; in four the needed passage was absent, and Q15 is the exception discussed above. The RAG prompts also instruct the model to acknowledge gaps, so this difference reflects prompt and context together.

Failures Across the Pipeline

Failure	Diagnostic	Possible response
Missing retrieval	Top-k lacks required facts	Expand query/k or retrieve another source
Incorrect retrieval	Related wording, wrong content	Try hybrid search/reranking and specific terms
Irrelevant citation	Existing ID does not support claim	Check claim against the cited passage
Beyond evidence	Claim exceeds supplied context	Separate supported facts and abstain on gaps

Table 5: Project synthesis of the seminar categories and [2].

One response may exhibit several failures. When retrieval is incomplete, the model may abstain, answer correctly from parametric memory, or fabricate details. Consequently, not every wrong answer is a retriever failure, and correctness alone does not establish grounding. A statement supported by the full PDF but absent from the actual prompt still exceeds the evidence supplied in that call.

Observed coverage of the failure categories. Missing and incorrect retrieval have clear observed cases (Q06, Q13), and Q09 shows partial retrieval. Beyond-evidence generation has one clear case, MiniLM's Q07 inference from a URL. We did not record a clear irrelevant citation; the nearest case is MiniLM's Q15, which cites a chunk for a mistaken explanation of missing evidence. The review has no separate citation-support label, so this absence is not evidence that citations are reliable.

Constructed citation example. Attaching `s192-p1-t336` to Q08's 97%/96% coverage claim is inappropriate because those values occur in `s192-p2-t336`. The existence of an ID does not establish support. This example illustrates a diagnostic and is not counted as an observed Gemini error.

Failures before generation. PDF extraction can omit text, and an index may retain outdated content. Tables can lose row/column structure. Chunk boundaries may remove a condition at the end

of a sentence or on the next page. IDF may reward matching terms without useful evidence; embeddings may prioritize broad topical similarity. A prompt may lose metadata, be truncated, or contain instructions embedded in a retrieved document. The demo delimits SOURCE blocks and treats them as evidence rather than instructions, but that instruction is not an absolute security guarantee.

Correctness is assessed against the reference answer, groundedness against the actual supplied context, and citation support against the specific cited passage. An abstention may be cautious when context is missing, but it is not a complete answer to an answerable question.

These distinctions suggest separate improvements. Better extraction addresses damaged evidence; retrieval improvements address selection; prompting may improve abstention and citation behavior; claim checking addresses unsupported conclusions. Improving only one stage cannot ensure end-to-end reliability. All proposed remedies would need new controlled measurements before being described as effective.

Answering the Research Question

Incorrect answers fell from 9/20 closed-book to 0/40 across both RAG conditions, so retrieval clearly helped in this run. This does not show that hallucination was eliminated. All questions were answerable from one short paper, none was unanswerable, answers were capped at 100 words, and each condition ran once. Residual problems also remain: MiniLM's Q07 goes beyond explicit evidence, its Q15 gives a mistaken reason for abstaining, and five responses abstain on answerable questions. Our answer is therefore qualified: supplying documents reduced wrong answers here, but this demonstration cannot establish elimination, and support still has to be checked claim by claim.

Proposed Verification Process

Split a response into checkable claims. For each, verify that its citation ID appears in the prompt, that the cited passage supports it, and that the source's conditions and scope are preserved. Multi-part questions need evidence for each part. If support is incomplete, provide the supported portion and identify what remains unknown. A model's self-explanation or self-grading is not independent confirmation.

Hybrid retrieval, reranking, or larger k may help when evidence is weak, but they should be tuned on a separate validation set. Cosine is not a probability, so an arbitrary similarity threshold does not guarantee reliability. Increasing k also adds noise, context cost, and material to verify; evaluation should include both quality and latency.

Document Updates and Provenance

Keep each source's URL, license, version, and hash. When a hash changes, repeat extraction, chunking, and indexing; remove stale chunks when a source is deleted. Run regression checks on answers and citations, retaining earlier snapshots for reproduction. Historical questions about the 2022 paper must remain linked to that snapshot rather than silently using current documentation. Metadata distinguishes the origin of evidence from explanations produced by the model.

Limitations

The study uses one paper, 20 deliberately authored questions, one small embedding model, and one generation per condition. Three of seven corpus pages mainly contain references, unlike a large store with competing sources. Gold spans do not cover all equivalent evidence, and page-wise splitting can interrupt context. The labels were assisted rather than independently scored by blinded reviewers.

There is no independent tuning/test split or statistical significance test. The public paper may have appeared in Gemini's training data, so closed-book performance is not a measure of entirely unseen knowledge. Temperature 0 does not guarantee identical future API outputs. Provider-side changes, quota, and network failures affect reproducibility. Only recorded successful responses are evaluated; API errors are not scored as wrong answers or replaced with invented responses.

A possible extension to an organizational document chatbot is outlined in Appendix B; it is a design proposal that was not implemented or measured.

5 Conclusion

RAG provides a mechanism to retrieve and cite evidence. In this run it reduced incorrect answers from 9/20 to 0/40, but that result does not establish that supplying documents eliminates hallucination. TF-IDF scores higher than MiniLM on the aggregate measurements here, but the answer gap is only 2 of 20 questions, no significance test was run, and individual questions show the reverse. Reliability requires inspecting sources, retrieval, and individual generated claims. A citation alone is insufficient. This demonstration illustrates failure mechanisms without proving a universal method ranking or a hallucination-free model.

AI disclosure. Authors Phan Anh Khoa (24125059) and Bùi Trọng Thành (24125043) are responsible for the submitted work. Codex assisted with experimental design, code, writing, and evaluation annotations; Claude Code assisted with report review and editing; Gemini generated the recorded experimental responses. Phan Anh Khoa and Bùi Trọng Thành each reviewed and accepted the assisted evaluation. No blind multi-rater assessment is claimed.

References

[1] P. Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems 33 (NeurIPS 2020). https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[2] J. Schneider (2024). *Explainable Generative AI (GenXAI): a survey, conceptualization, and research agenda*. Artificial Intelligence Review 57, 289. https://doi.org/10.1007/s10462-024-10916-x.

[3] J. Zhu et al. (2022). *abess: A Fast Best-Subset Selection Library in Python and R*. JMLR 23, 1–7. CC BY 4.0. https://jmlr.org/papers/v23/21-1060.html.

[4] F. Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html. Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[5] Sentence Transformers. *all-MiniLM-L6-v2 model card*. Apache 2.0. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

[6] Google. *Gemini API models*. https://ai.google.dev/gemini-api/docs/models. Actual model and request metadata are recorded in the logs.

[7] TorchMetrics. *Retrieval Recall*. https://lightning.ai/docs/torchmetrics/stable/retrieval/recall.html.

[8] Z. Rackauckas (2024). *RAG-Fusion: a New Take on Retrieval-Augmented Generation*. https://arxiv.org/abs/2402.03367.

[9] S.-Q. Yan et al. (2024). *Corrective Retrieval Augmented Generation*. https://arxiv.org/abs/2401.15884.

[10] L. Medrano, A. Verma, and M. Chhabra (2026). *Scaling Retrieval Augmented Generation with RAG Fusion: Lessons from an Industry Deployment*. https://arxiv.org/abs/2603.02153.

Appendix A: Reproduction Materials

Repository: https://github.com/Khoaphan-hub/seminar-rag.

- data/questions.json: 20 questions, references, and gold evidence.
- results/retrieval_summary.csv: aggregate retrieval measurements.
- results/retrieval_details.json: per-question scores and chunks.
- results/generation.jsonl: prompts and successful API responses.
- docs/EVALUATION.md: rubric and annotation limitations.
- docs/PRESENTATION_AND_QA.md: team roles and discussion questions.

The source PDF and license information accompany the code. API keys, Python environments, model weights, and the old vector store are excluded from Git. Instructor-supplied foundational papers remain local and are not included under a blanket repository license.

Appendix B: Extension to an Organizational Document Chatbot

An organizational chatbot would need to handle more documents, versions, and ambiguous questions than this seven-page study. A proposed extension is: ingest versioned documents; normalize and, when useful, rewrite a question into focused subqueries; retrieve candidates with lexical and dense search; merge and rerank candidates; assess whether each requested fact has evidence; then generate a cited answer or identify the unsupported part. Feedback and document changes would trigger review and reindexing. These are design options, not features implemented or measured in this project.

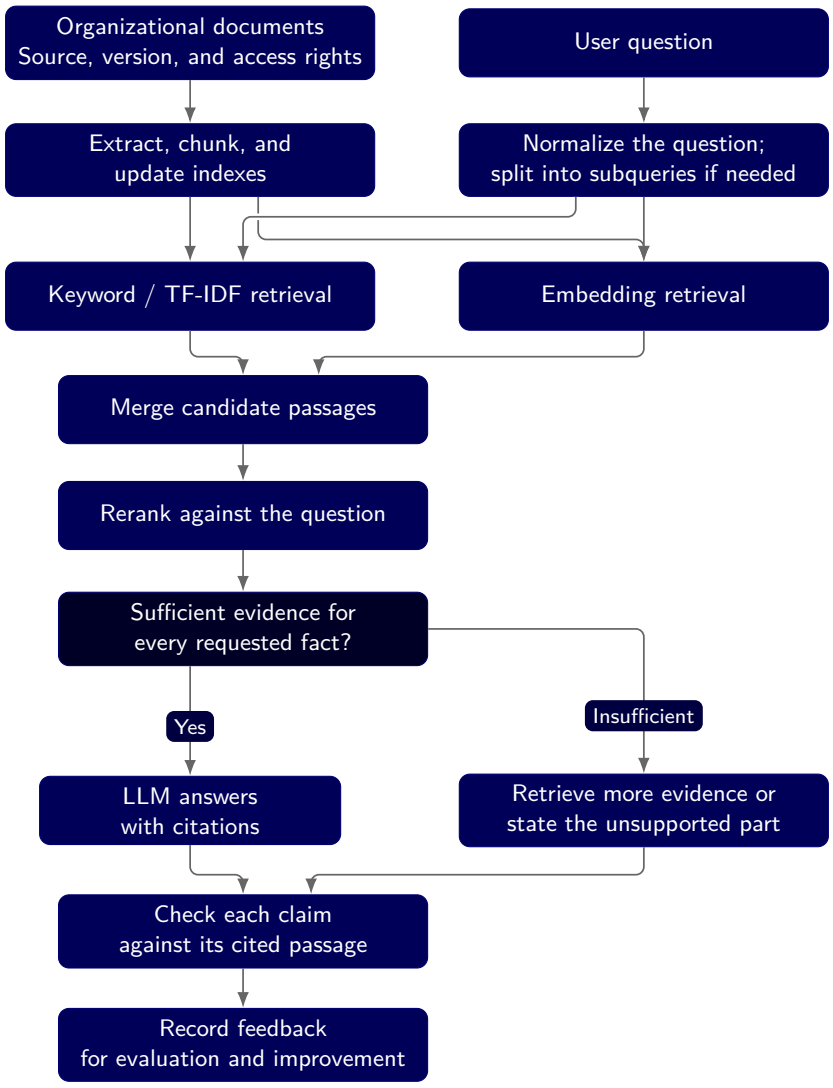


Figure 2: Proposed organizational RAG pipeline, informed by multi-query retrieval [8] and retrieval correction [9]. These additional stages have not been implemented or evaluated in this project.

The observed failures motivate these options: alternate wording for Q06's version requirements, reranking for Q13's architecture passage, and separate license subqueries for Q09. Multi-query fusion [8] and corrective retrieval [9] inform this proposal, but do not establish gains on our corpus. An industry study reports that reranking and context truncation can erase retrieval gains [10]; final answer support and latency therefore also need evaluation. Access controls and source versions must be preserved. This is our synthesis, not a universal industry architecture.